

ADL Homework 3 — Feedforward Neural Networks on MNIST

Applied Deep Learning, Spring 2026

We implement a two-layer fully connected network in PyTorch (based on the Lecture 5 demo), train it on MNIST, study randomness and validation-based model selection, perform a hyperparameter grid search, and visualize learned features with t-SNE.

Device: cpu

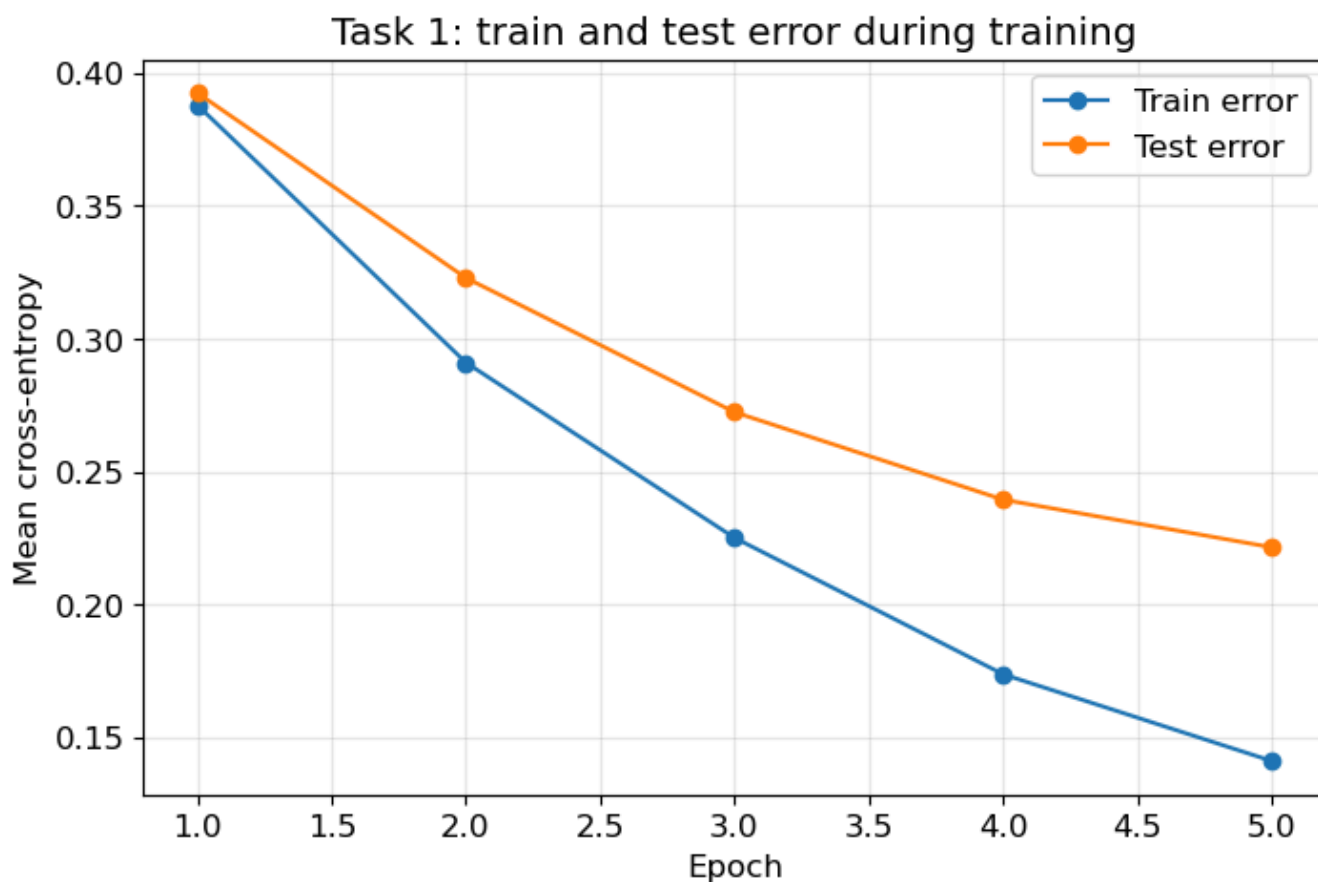
Hyperparameters

We follow the Lecture 5 coding demo (`lecture_code/test_torch.py`), with the training set reduced for faster CPU training (allowed by the assignment).

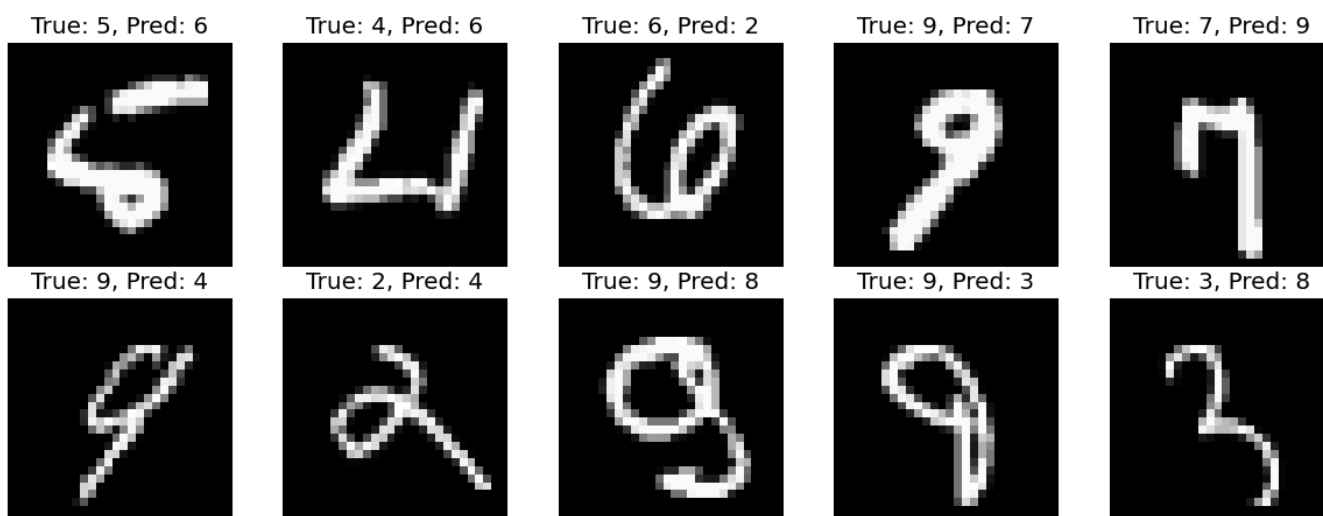
Parameter	Value
Architecture (Tasks 1-3)	784 → 500 → ReLU → 10
Loss	Cross-entropy (mean over dataset, per epoch)
Optimizer	Adam (weight decay 0)
Learning rate (Tasks 1-3)	0.001
Batch size (Tasks 1-3)	100
Epochs (all tasks)	5
Training subset	6,000 samples (from MNIST train, after val split)
Validation set (Tasks 3-4)	10,000 samples (held out from original 60k train)
Test set	10,000 samples (standard MNIST test)
Seed (Task 1)	0
Seeds (Tasks 2-3)	0, 1, 2, 3, 4
Validation split seed	42
Task 4 grid	hidden ∈ {256, 500, 768}, batch ∈ {50, 100}, lr ∈ {5e-4, 1e-3, 5e-3} (18 models)

Task 1 — MNIST classification

Epoch 1/5 | e_tr=0.3880 | e_te=0.3925
 Epoch 2/5 | e_tr=0.2910 | e_te=0.3229
 Epoch 3/5 | e_tr=0.2250 | e_te=0.2723
 Epoch 4/5 | e_tr=0.1736 | e_te=0.2394
 Epoch 5/5 | e_tr=0.1410 | e_te=0.2214
 Final test error: 0.2214



Task 1: misclassified test images

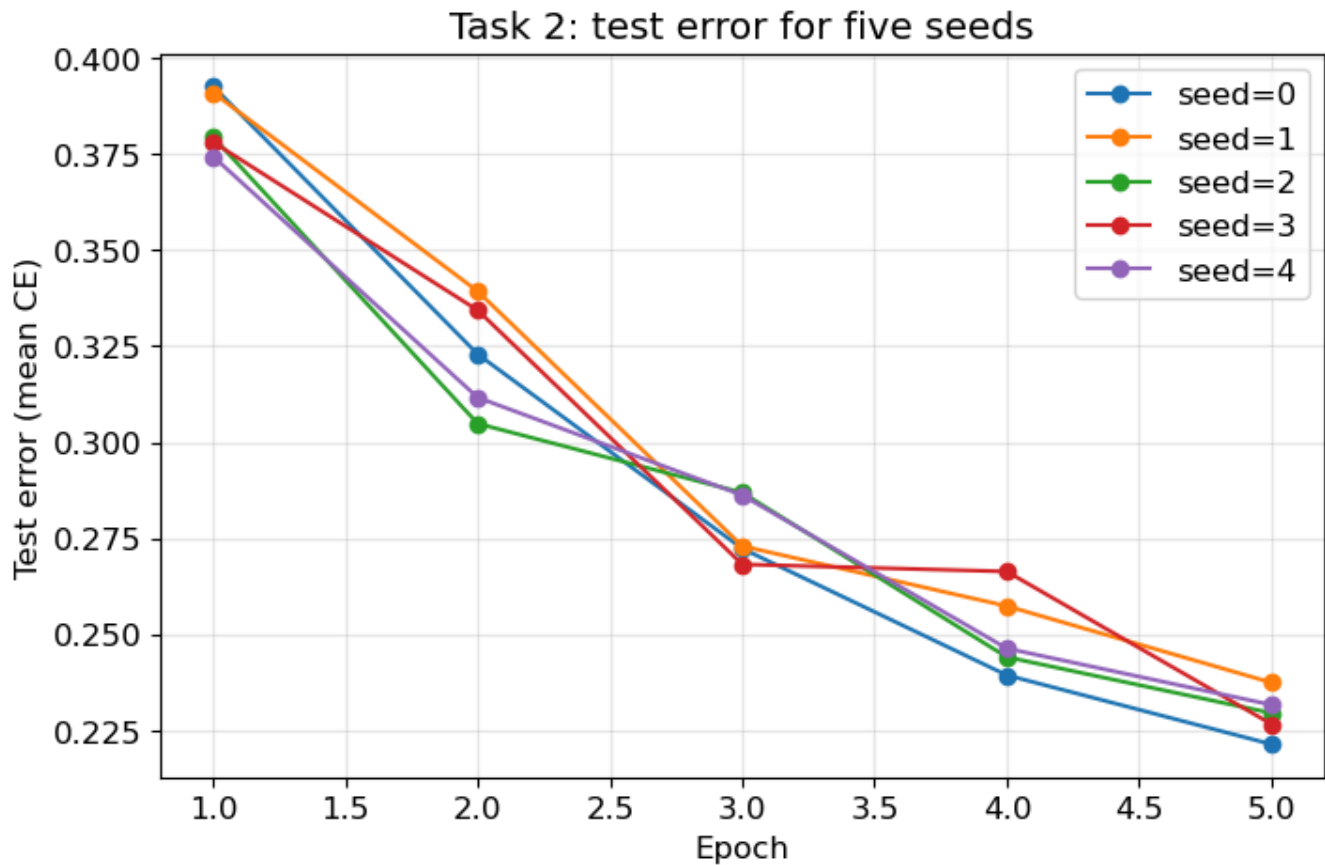


Task 2 — Mitigating pseudorandomness

We fix seeds for Python, NumPy, PyTorch, and the DataLoader generator so repeated runs with the same seed are identical. We then train with five different seeds and compare test-error curves.

Mean final test error: 0.2294

Std final test error: 0.0053



Discussion (Task 2)

We trained with five seeds on a 6,000-sample training subset (allowed by the assignment for faster CPU training). The final test errors were 0.2214, 0.2375, 0.2295, 0.2267, and 0.2317, giving a mean of **0.2294** and standard deviation of **0.0053**.

All five test-error curves decrease steadily and have very similar shape. The small standard deviation (about 2.3% of the mean) indicates that the model is **relatively robust** to weight initialization and mini-batch ordering: changing the seed shifts the final error only slightly, without changing the overall training behavior.

Task 3 — Validation dataset

We hold out 10,000 samples from the original MNIST training set as validation. For each of five seeds we report the test error at the epoch with minimum validation error: (e_{te}^*, e_{va}^*) .

seed	e*_te	e*_va	best_epoch
0	0.2384	0.2662	5
1	0.2486	0.2640	5
2	0.2342	0.2577	5
3	0.2307	0.2488	5
4	0.2319	0.2530	5

Mean e*_te: 0.2368

Mean e*_va: 0.2580

The test set is **not** used for model selection — only validation error determines the chosen epoch.

Task 4 — Grid search

We search over hidden layer size, batch size, and learning rate. Each configuration is evaluated with the validation procedure from Task 3.

Training 18 models...

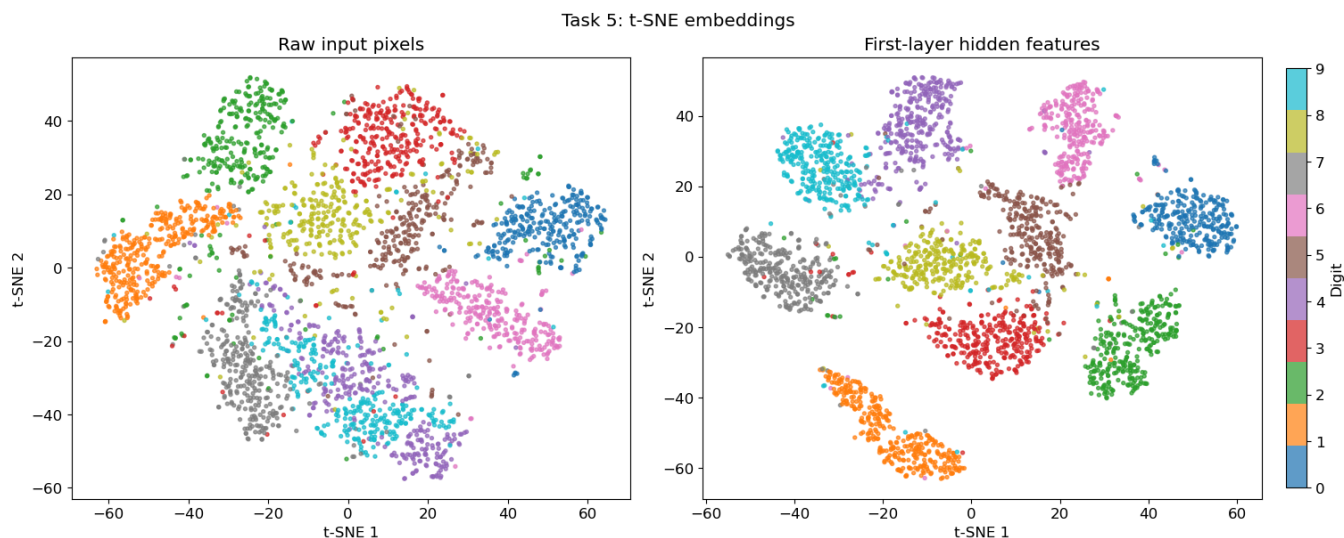
```
{'hidden_size': 256, 'batch_size': 50, 'learning_rate': 0.0005} -> e*_va=0.3058, e*_te=0.2778 (epoch 5)
{'hidden_size': 256, 'batch_size': 50, 'learning_rate': 0.001} -> e*_va=0.2600, e*_te=0.2331 (epoch 5)
{'hidden_size': 256, 'batch_size': 50, 'learning_rate': 0.005} -> e*_va=0.2162, e*_te=0.1910 (epoch 5)
{'hidden_size': 256, 'batch_size': 100, 'learning_rate': 0.0005} -> e*_va=0.3320, e*_te=0.3057 (epoch 5)
{'hidden_size': 256, 'batch_size': 100, 'learning_rate': 0.001} -> e*_va=0.2903, e*_te=0.2615 (epoch 5)
{'hidden_size': 256, 'batch_size': 100, 'learning_rate': 0.005} -> e*_va=0.2192, e*_te=0.1936 (epoch 5)
{'hidden_size': 500, 'batch_size': 50, 'learning_rate': 0.0005} -> e*_va=0.2771, e*_te=0.2510 (epoch 5)
{'hidden_size': 500, 'batch_size': 50, 'learning_rate': 0.001} -> e*_va=0.2321, e*_te=0.2075 (epoch 5)
{'hidden_size': 500, 'batch_size': 50, 'learning_rate': 0.005} -> e*_va=0.2093, e*_te=0.1880 (epoch 2)
{'hidden_size': 500, 'batch_size': 100, 'learning_rate': 0.0005} -> e*_va=0.3048, e*_te=0.2777 (epoch 5)
{'hidden_size': 500, 'batch_size': 100, 'learning_rate': 0.001} -> e*_va=0.2662, e*_te=0.2384 (epoch 5)
{'hidden_size': 500, 'batch_size': 100, 'learning_rate': 0.005} -> e*_va=0.2099, e*_te=0.1760 (epoch 4)
{'hidden_size': 768, 'batch_size': 50, 'learning_rate': 0.0005} -> e*_va=0.2547, e*_te=0.2295 (epoch 5)
{'hidden_size': 768, 'batch_size': 50, 'learning_rate': 0.001} -> e*_va=0.2142, e*_te=0.1932 (epoch 5)
{'hidden_size': 768, 'batch_size': 50, 'learning_rate': 0.005} -> e*_va=0.2205, e*_te=0.1923 (epoch 4)
{'hidden_size': 768, 'batch_size': 100, 'learning_rate': 0.0005} -> e*_va=0.2836, e*_te=0.2568 (epoch 5)
{'hidden_size': 768, 'batch_size': 100, 'learning_rate': 0.001} -> e*_va=0.2476, e*_te=0.2198 (epoch 5)
{'hidden_size': 768, 'batch_size': 100, 'learning_rate': 0.005} -> e*_va=0.1965, e*_te=0.1616 (epoch 4)
```

hidden	batch	lr	e*_te	e*_va	epoch	
768	100	0.0050	0.1616	0.1965	4	<-- best
500	50	0.0050	0.1880	0.2093	2	
500	100	0.0050	0.1760	0.2099	4	
768	50	0.0010	0.1932	0.2142	5	
256	50	0.0050	0.1910	0.2162	5	
256	100	0.0050	0.1936	0.2192	5	
768	50	0.0050	0.1923	0.2205	4	
500	50	0.0010	0.2075	0.2321	5	
768	100	0.0010	0.2198	0.2476	5	
768	50	0.0005	0.2295	0.2547	5	
256	50	0.0010	0.2331	0.2600	5	
500	100	0.0010	0.2384	0.2662	5	
500	50	0.0005	0.2510	0.2771	5	
768	100	0.0005	0.2568	0.2836	5	
256	100	0.0010	0.2615	0.2903	5	
500	100	0.0005	0.2777	0.3048	5	

256	50	0.0005	0.2778	0.3058	5
256	100	0.0005	0.3057	0.3320	5

Task 5 — Feature analysis (t-SNE)

We compare 2D t-SNE embeddings of raw pixel inputs x_i and first-layer hidden features $z_i = \sigma((W^{(1)})^\top x_i + b^{(1)})$ from the best grid-search model.



Discussion (Task 5)

We applied t-SNE to 3,000 test samples using the best Task 4 model (hidden size 768). In the **raw pixel** embedding, several classes overlap heavily — especially digits 4, 7, 8, and 9 in the central region — because similar stroke patterns look alike in 784-dimensional pixel space. Some structure is still visible (e.g., digit 1 on the left, digit 0 on the right), but boundaries between classes are fuzzy.

In the **first-layer hidden features** $z_i = \sigma((W^{(1)})^\top x_i + b^{(1)})$, the same digits form much tighter, more separated clusters. Digits that overlapped in raw space (such as 7 and 9) are now far apart, and digit 1 splits into two compact groups, likely reflecting different handwriting styles. This shows the network learned a more discriminative representation: the hidden layer reorganizes pixel information into features that make different classes easier to separate, which supports the improved test performance observed in Task 4.